SOFTWARE CONSTRUCTION & DEVELOPMENT

COMPLETE MIDTERM EXAM PREPARATION GUIDE — FULL SYLLABUS (WEEKS 1–8)

Course Instructor: Engr. Muhammad Umer Haroon | **Institute:** IMSciences | **Target:** BSSE Midterm Examination

MODULE 1: SOFTWARE CONSTRUCTION VS. DEVELOPMENT & CLEAN CODE (LECTURE 1)

1.1 Software Construction vs. Development

Software Development is the umbrella process encompassing all software lifecycle phases (requirements, architecture, design, construction, testing, deployment, maintenance). **Software Construction** is the central hands-on activity focusing on detailed creation through coding, debugging, testing, and component integration.

1.2 The 5 Construction Fundamentals

Every construction activity must adhere to five fundamental principles:

- **Minimizing Complexity:** The primary technical driver in construction. Keeping code simple so human brains can comprehend it.
- **Anticipating Change:** Writing flexible, modular code because software requirements inevitably evolve.
- **Constructing for Verification:** Designing code so that bugs can be easily caught via testing and assertions.
- **Reuse:** Leveraging existing well-tested code, functions, and libraries to reduce effort and defect rates.
- **Standards in Construction:** Adhering to team coding conventions, standard layouts, and naming practices.

1.3 Clean Code Principles & Naming Rules

As defined by Michael Feathers: *'Clean code always looks like it was written by someone who cares.'* Clean code is readable, maintainable, elegant, and free of gratuitous complexity.

Intent-Revealing Naming Rule: A variable, function, or class name must answer three big questions: *Why it exists, what it does, and how it is used.* If a name requires a comment to explain itself, the name fails to reveal its intent.

```
Bad: int d; // elapsed time in days
Good: int elapsedTimeInDays; int daysSinceCreation; int fileAgeInDays;
```

> **EXAM TIP — LECTURE 1:** Be ready to explain why int d; is poor clean code. Answer: It reveals nothing about elapsed time or units, forcing developers to read comments. Replacing it with elapsedTimeInDays eliminates the need for comments entirely.

MODULE 2: PYTHON COLLECTIONS, FUNCTIONS & REUSABILITY (LECTURE 2)

2.1 Python Collections & The Set Trick

Python provides four standard collection types with distinct duplicate and mutability rules:

Collection	Ordered?	Mutable?	Duplicates?	Syntax Example
List	Yes	Yes	Yes	['cd70', '125', 'GS150']
Tuple	Yes	No (Immutable)	Yes	('cd70', '125', 'GS150')
Dictionary	Yes (Modern)	Yes	Keys: No / Values: Yes	{'id': 1, 'name': 'Ali'}
Set	No (Random)	Yes (Add/Remove)	**No (Discards extra)**	{'cd70', '125', 'GS150'}

Sir Umer's Set Evaluation Trick: In Python, True == 1 and False == 0 under the hood. When evaluating a set literal containing both booleans and integers:

`example_set2 = {True, 1, 1, 0, 0, True, False}` → **Output:** {0, True} (or {False, 1} depending on hash order, discarding duplicate 1s and 0s).

2.2 Routine vs. Function vs. Procedure vs. Method

While modern developers often mix up these terms, clean software architecture requires understanding their technical distinctions:

Term	Returns Value?	Primary Purpose	Caller Usage	Language Context
Routine	Varies	Umbrella term for any callable code block	Called by name	Older languages (BASIC, Fortran)
Function	**Yes (Always)**	Compute and return a calculated result	Assigns or uses return value	Pascal, C, Python (`def`)
Procedure	**No (Returns void/None)**	Perform an action or side-effect	Called as standalone statement	Pascal, Ada, VB
Method	Varies	Routine bound to an OOP Class / Object	Called on object (`obj.method()`)	OOP languages (Python `self`)

Python Unification: In Python, every block defined with def is called a function, returning None if no return statement is specified. However, the **design distinction remains vital**: developers must consciously decide if a routine computes a value or executes an action.

2.3 Why Create Routines? (The Technical Reasons)

Textbooks often claim avoiding duplication is the main reason to write routines. In Software Construction, the **primary reason is to REDUCE COMPLEXITY** through abstraction and information hiding. Other core reasons include:

- **Reduce Complexity:** Hiding complex operational logic behind a single callable abstraction.
- **Avoid Duplicated Code:** Centralizing shared logic so bug fixes and updates happen in one place.
- **Hide Pointer Operations:** Isolating pointer/memory arithmetic so calling code remains clean.
- **Improve Portability:** Encapsulating OS-specific or hardware-dependent calls in dedicated routines.
- **Improve Performance:** Centralizing critical code for easy profiling, optimization, and caching.

2.4 Golden Rules of Clean Functions

1. **Small Functions:** Functions should be small (ideally fitting on a single screen).
2. **Do One Thing:** A function should do one thing, do it well, and do it only.
3. **Shallow Indentation:** Indent level inside a function **should not exceed 1 or 2 levels**.
4. **Ward's Principle:** Clean code is when each routine turns out to be pretty much what you expected.

MODULE 3: FUNCTION ARGUMENTS, SCOPE & RECURSION (LECTURE 2B)

3.1 Function Arguments & Testing Complexity

Clean Code classifies parameter counts as follows:

Argument Count	Formal Name	Clean Code Rating	Testing Impact
0 arguments	Niladic	Ideal / Excellent	Trivial to test
1 argument	Monadic	Very Good	Straightforward combinations
2 arguments	Dyadic	Good / Acceptable	Manageable test matrix
3 arguments	Triadic	Avoid where possible	Testing combinations becomes daunting
4+ arguments	Polyadic	Requires special justification	Exponential explosion of test cases

Argument Objects: When a function needs 3+ parameters, group related concepts into an object: e.g., refactoring makeCircle(x, y, radius) into makeCircle(center_point, radius).

3.2 Structured Programming Rules & Scope

• **Dijkstra's Single Entry / Exit Rule:** Every function and code block should have **one entry point and one exit point** (a single return statement, no break/continue/goto).
• **Nested Functions & Nonlocal Scope:** Python supports defining functions inside other functions. Variables in the outer function's local scope are called **nonlocal names** relative to the inner function.

3.3 Recursion vs. Iteration & Tail Call Optimization (TCO)

Recursion pushes a new frame onto the **Call Stack** for every recursive call. Deep recursion risks crashing with a RecursionError.

Tail Recursion: A recursive function where the recursive call is the **absolute last operation** performed before returning.

```
Non-Tail Recursive:
def factorial(n):
    if n == 0: return 1
    return n * factorial(n - 1) # Multiplication waits on stack!

Tail Recursive:
def tail_factorial(n, acc=1):
    if n == 0: return acc
    return tail_factorial(n - 1, n * acc) # Immediate return!
```

HIGH-YIELD EXAM QUESTION — DOES PYTHON SUPPORT TCO?
NO! Python does NOT optimize tail recursion. Even if you write a tail-recursive function, Python does not perform Tail Call Optimization (TCO) and will still hit the recursion limit (~1000 calls). Therefore, **iteration (loops) is strictly preferred over recursion in Python**.

MODULE 4: TECH STACK SELECTION & SOFTWARE ARCHITECTURE (LECTURE 3)

4.1 Tech Stack Layers & Common Examples

A **tech stack** is the specific set of technologies chosen across all system layers:

Layer	Role	Common Technologies
Front-end	Client-side user interface	React, Vue, HTML/CSS, Angular
Back-end	Server-side business logic	Python (Django/FastAPI), Node.js, Java, Go
Database	Data storage and query engine	PostgreSQL, MySQL, MongoDB, SQLite
Infrastructure	Deployment, cloud & hosting	Docker, Kubernetes, AWS, DigitalOcean
Support Tools	Quality and process automation	Git, CI/CD pipelines, PyTest, Monitoring

Popular Stacks: MERN (MongoDB, Express, React, Node), LAMP (Linux, Apache, MySQL, PHP), Django Stack (Python, Django, PostgreSQL), Spring Stack (Java, Spring Boot).

4.2 Stack Selection Criteria & Selection Mistakes

Selecting a stack involves evaluating: *Maturity/stability, community size, ease of hiring, fit for problem, load performance, security patching speed, running cost (cloud/licensing), and 5-year longevity.*

Common Selection Mistakes: Choosing trendy technology (Hype-driven development), individual developer bias, ignoring niche hiring costs, overengineering small tools for massive scale, and underestimating migration costs.

4.3 Build vs. Buy Decision (Bespoke vs. COTS / SaaS)

• **Golden Rule: BUILD** custom software for core capabilities that provide a unique competitive advantage. **BUY** commercial off-the-shelf (COTS) or SaaS software for standard utility functions (payroll, email, accounting).
• **Total Cost of Ownership (TCO):** Consider 5-year license, integration, customization, and vendor exit strategy costs.

MODULE 5: CONTINUOUS IMPROVEMENT, RELEASES & SEMVER (LECTURE 3)

5.1 Release Types & Release Cadences

A release is a packaged version snapshot. Release types: Major (breaking changes), Minor (backward-compatible features), Patch (bug/security fixes). Cadences include: Continuous Deployment, Fixed Schedule, Feature-based, and On-demand.

5.2 Deployment Safety: Rollback vs. Canary Release

• **Rollback:** Reverting production to a previous working version when a release causes issues.
• **Canary Release:** Deploying a new version to a small subset of users first to catch defects before full rollout.

5.3 Semantic Versioning (SemVer) & Patching Tension

SemVer format: **MAJOR . MINOR . PATCH** (e.g., 3.7.2):

Version Part	Increment Condition	Backward Compatible?	Risk Level
MAJOR (3.x → 4.0)	Incompatible API / breaking changes	**NO**	High (Requires migration)
MINOR (3.7 → 3.8)	New features added	Yes	Low / Moderate
PATCH (3.7.2 → 3.7.5)	Bug fixes & security patches	Yes	Very Low

Technical Bankruptcy: Continually delaying security patches creates severe vulnerability risks. Neglecting code maintenance leads to system decay where complete rewrites become unavoidable.

MODULE 6: REGULAR EXPRESSIONS (REGEX) & TEXT PROCESSING (LECTURE 4)

6.1 Regular Expressions & Python `re` Module

Regex is a mini-language for matching text patterns. Essential Python re functions:

• re.search(pat, str): Finds first occurrence; returns Match object or None.
• re.findall(pat, str): Returns all matches as a Python list.
• re.sub(pat, repl, str): Replaces matches with a new string.
• re.split(pat, str): Splits string at pattern occurrences.

6.2 Raw Strings (`r"..."`) in Python

Raw strings (prefixed with r) treat backslashes as literal characters without interpreting Python string escape sequences (e.g., r"\d+\s+" avoids double-escaping).

6.3 Metacharacters & Character Classes Summary

Symbol / Class	Meaning & Behavior	Example Pattern	Matches
.	Any single character except newline	c.t	cat, cot, c9t
^ / $	Start / End of string anchor	^Hello / dog.$	String start/end
*** / + / ?**	0+ / 1+ / Optional (0 or 1)	cats?	cat, cats
{m,n}	Between m and n repetitions	a{2,4}	aa, aaa, aaaa
\d / \D	Digit / Non-digit character	\d+	85, 100
\w / \W	Alphanumeric + '_' / Non-alphanumeric	\w+	Ali_123
\s / \S	Whitespace / Non-whitespace	\s+	Spaces, tabs
[...] / [^...]	Character set / Negated set	[A-Z][a-z]+	Ali, Sara
(?<=...) / (?=...)	Lookbehind / Lookahead assertion	(?<=@)\w+	Domain name after @

MODULE 7: DATA INTERCHANGE FORMATS & SERIALIZATION (LECTURE 5)

7.1 Serialization vs. Deserialization

• **Serialization (Furniture Disassembly):** Converting in-memory objects (dicts, classes) into a byte/string sequence for storage or network transmission.
• **Deserialization (Furniture Reassembly):** Reconstructing in-memory data structures from a serialized byte/string sequence.
• **Pair Rule:** Serialization and deserialization always operate as a pair; both systems must agree on the format.

7.2 JSON vs. XML Comparison

Feature / Metric	JSON (JavaScript Object Notation)	XML (eXtensible Markup Language)
Data Model	Key-value pairs and arrays	Nested tags and attributes
Syntax Rules	Double quotes for keys/strings, lowercase booleans	Matching opening/closing tags, single root
Comment Support	**NO COMMENTS ALLOWED** (Invalid syntax)	**Supported** (<!-- comment -->)
Schema Support	JSON Schema (Optional)	XSD (Standardized Enterprise Schema)
Namespaces	Not supported	Supported (xmlns)
Parsing & Size	Smaller payload, fast parsing	Verbose, larger size, slower parsing
Primary Use	Web APIs, Browser communication	Legacy enterprise systems, documents

Python JSON Module: json.dumps()/loads() for strings; json.dump()/load() for file objects.

7.3 Modern Binary & Config Formats

• **YAML:** Human-readable config format using indentation. Supports comments. Ideal for DevOps (Kubernetes, Docker).
• **Protocol Buffers (Protobuf):** Google binary format using .proto schema. High speed, compact payloads for microservices/gRPC.
• **Apache Avro:** Binary format storing schema with data. Supports schema evolution in Big Data (Kafka, Hadoop).
• **Cap'n Proto:** Extreme speed with zero-copy deserialization.

MODULE 8: DEFENSIVE PROGRAMMING & EXCEPTION HANDLING (LECTURE 6)

8.1 Defensive Programming & The 3 Enemies

Defensive programming anticipates failures before they occur so software fails gracefully rather than crashing. The **Three Enemies of Reliable Software** are:

1. **Bad Input:** Users entering invalid types or out-of-range values (e.g. passing string to math function).
2. **Bad Environment:** External resources failing (missing files, dropped networks, full disk).
3. **Bad Assumptions:** Developer assuming conditions that aren't guaranteed (e.g. assuming list is never empty).

8.2 The 3 Layers of Defense (Airport Analogy)

Layer	Primary Purpose	Airport Analogy
1. Input Validation	Reject bad data **at the boundary** before internal logic	**Security Check at Door:** Bad items stopped before entering
2. Internal Checks / Assertions	Verify internal invariants that must hold true in correct code	**Equipment Maintenance:** Checking systems before arrival
3. Exception Handling	Recover from unpreventable runtime environmental failures	**Emergency Exit Plan:** Managing unexpected events inside

Boundary Validation Rule: Validate data *once as it enters* at the boundary (API, user input, file read), then trust it throughout internal functions.

8.3 Exception Handling Blocks & Anti-Patterns

• try: Risky code that might fail.
• except SpecificError: Handles expected failures.
• else: Runs ONLY if no exception occurred in try.
• finally: Runs ALWAYS for resource cleanup (closing files/connections).

CRITICAL CODE REVIEW TRAP — BARE EXCEPTIONS:
Writing except Exception: pass is a dangerous anti-pattern! It silently swallows all errors, hiding real logic bugs in your code instead of letting unexpected bugs surface to be fixed. Always catch specific exceptions (e.g., except FileNotFoundError:).

MODULE 9: COURSE ASSIGNMENTS, PROJECT EXPECTATIONS & EXAM SUMMARY

9.1 Assignment 1 & Agile Project Guidance

• **Git Collaboration:** Shared public repository, clear commit messages, version tags (v0.1.0 for initial, v1.0.0 for first release). No saving multiple file copies for versions.
• **Agile Requirements:** Lightweight requirements document, backlog tracking, README.md status overview.

9.2 Case Study: CodeInspector Project (Abdul Samad)

An AST-based static code evaluator reviewing student Python projects against clean code rules without executing target code: *Function size (≤20 lines), shallow nesting (≤depth 2), parameter limit (≤2 explicit args), PEP 8 naming/formatting, and completeness checks.*

QUICK SUMMARY FOR EXAM DAY REVISION
1. Construction Fundamentals: Minimizing complexity, anticipating change, verification, reuse, standards.
2. Clean Naming: Names reveal intent (why, what, how); eliminate comments.
3. Function Rules: Small, do one thing, max 1-2 indent levels.
4. Routines: Function (returns value) vs Procedure (side-effect, void/None) vs Routine vs Method (self).
5. Arguments: Niladic (0) > Monadic (1) > Dyadic (2). Wrap 3+ args in Argument Objects.

6. Recursion: Python does NOT optimize tail recursion (no TCO); stack limit ~1000. Prefer loops.
7. Tech Stack: Build core competitive advantage; Buy common COTS/SaaS utility tools.
8. SemVer: MAJOR (breaking), MINOR (feature), PATCH (fix). Canary release tests small group first.
9. Regex: re.findall(), raw strings r'...', character classes \d,\w,\s, lookarounds (?<=@)\w+.
10. Formats: JSON (no comments, double quotes) vs XML (comments, XSD) vs Protobuf (gRPC binary).
11. Defensive Rules: Validate at boundary only. Catch specific exceptions; never swallow errors with bare except:.